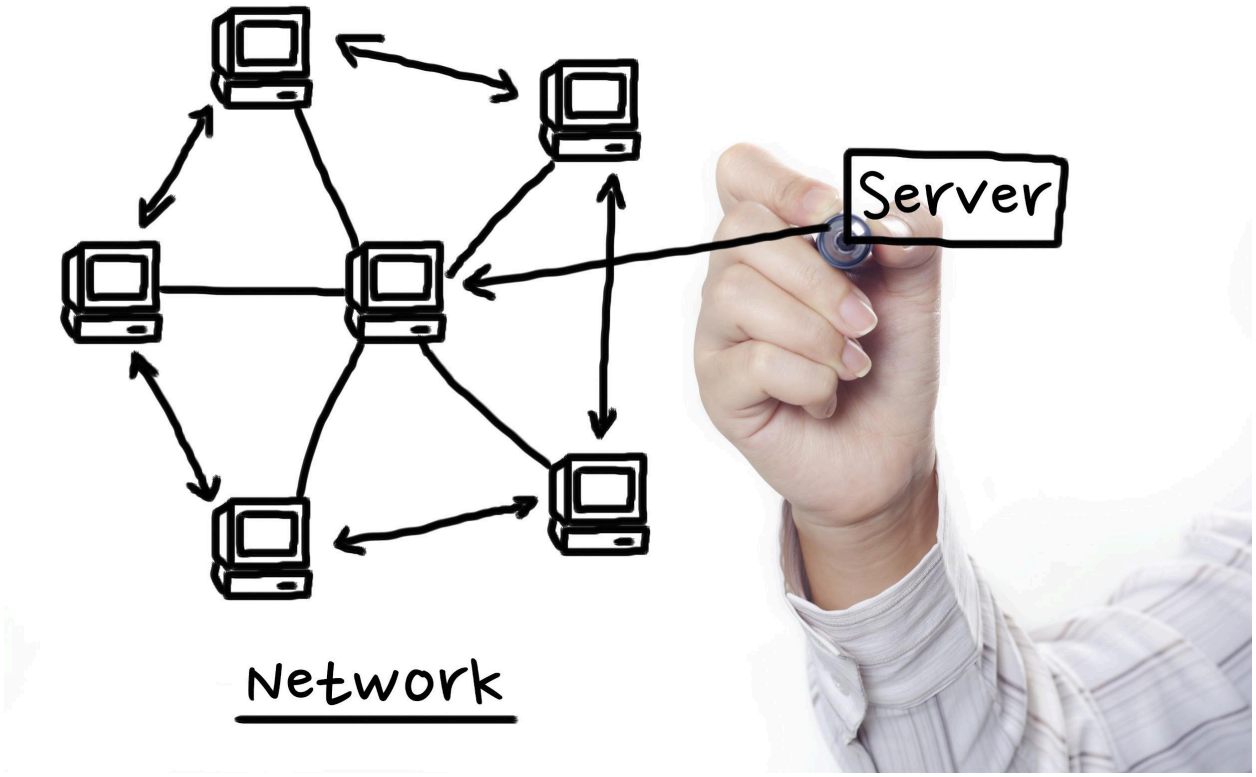
I. High-Level Concept & Use Case

The Pitch: "Most messengers rely on the Cloud (Internet). If the internet is cut, communication dies. **SecureLink** is a decentralized, military-grade communication suite that operates entirely on a Local Area Network (LAN). It features automatic peer discovery, AES-256 encryption, and file transport, designed for secure, internet-denied environments."

II. The System Architecture

We are building a **Hybrid Client-Host System**. This means every version of the app contains *both* Server and Client capabilities.



Shutterstock

1. The Logical Layers (The Stack)

You must divide your application into 3 distinct layers. This is how you will explain it in your project report.

- **Layer 1: The Presentation Layer (UI)**

    - **Technology:** `CustomTkinter` (Python).
    - **Role:** Handles user inputs (clicks, typing) and displays data. It **never** talks to the network directly. It sends commands to the Logic Layer.
- **Layer 2: The Logic/Controller Layer**
    - **Technology:** Python Functions & `Threading`.
    - **Role:** The brain. It decides: "Is this a file or text?", "Should I encrypt this?", "Is this a self-destruct message?"
- **Layer 3: The Network & Data Layer**
    - **Technology:** `socket` (TCP/UDP), `sqlite3`, `cryptography`.
    - **Role:** The muscle. It chops data into bytes, sends them over wires, and saves history to the disk.

2. The Network Protocol (The "Language")

To separate yourself from amateurs, you will design a **Custom Application Protocol**. You won't just send strings; you will send **Packets**.

Every piece of data sent over the wire will follow this strict format: `[HEADER] + [SEPARATOR] + [PAYLOAD]`

- **Header (Fixed Size):** Contains the "Meta-Data."
    - `TYPE`: What is this? (`TXT` = Text, `FIL` = File, `DSC` = Discovery, `AUT` = Auth).
    - `ENC`: Is it encrypted? (`1` = Yes, `0` = No).
    - `SIZE`: How big is the payload? (e.g., `00450` bytes).
- **Payload:** The actual data (The text message or the image bytes).

III. The Tech Stack (What to Use)

You don't need heavy frameworks. You need raw control.

- **Language:** Python 3.10+
- **GUI Engine:** `CustomTkinter` (Looks modern/professional).
- **Network Core:** `socket` (Standard library), `ipaddress` (For IP validation).
- **Concurrency:** `threading` (To keep GUI alive), `queue` (To manage message flow).
- **Security:** `cryptography` (specifically `Fernet` for symmetric encryption).
- **Deployment:** `PyInstaller` (To compile to .exe).
- **Icons/Assets:** `Pillow` (PIL) for image handling in UI.

IV. Detailed Module Breakdown (Who Does What)

Here is the exact division of labor for your 3 members.

Member 1: The Architect (UI & Integration)

Focus: The "Face" and the "Database".

1. **UI Design:** Build the Dashboard.
    - Sidebar: List of active users (found via Auto-Discovery).
    - Main Area: Chat bubble interface (Green for me, Gray for others).
    - Status Bar: "Connected to [IP] | Encryption: ON".
2. **Local Storage:** Implement `sqlite3`.
    - Create a table `chat_history` (Sender, Message, Timestamp).
    - Load previous chats when the app opens.
3. **The "Self-Destruct" Timer:**
    - Write a UI function that displays a message, waits 10 seconds, and then deletes the text widget content from the screen.

Member 2: The Network Engineer (Transport)

Focus: Connectivity and "The Magic".

1. **TCP Engine (The Tunnel):**
    - Create the `Server` class (Listen for connections).
    - Create the `Client` class (Connect to IP).
2. **UDP Beacon (The Auto-Discovery):**
    - **Broadcaster:** A background thread that shouts "I AM HERE" to port 5556 every 2 seconds.
    - **Listener:** A background thread that listens on port 5556 and updates the "Active Users" list in the UI when it hears a shout.
3. **Firewall Handling:** Research how to programmatically ask Windows for "Network Permission" so the user doesn't have to struggle.

Member 3: The Data Engineer (Security & Protocol)

Focus: The Packet Structure and File Logic.

1. **The Packetizer:** Write the function that wraps data.
    - Input: "Hello" -> Output: `TXT|0005|Hello`.
    - Input: (Image) -> Output: `FIL|4096|...binary...`.
2. **The Encryption Vault:**
    - Generate a Key.
    - `encrypt(message)` -> returns scrambled bytes.
    - `decrypt(message)` -> returns original text.
3. **File Chunking:**

    - Write the loop that reads a file 4KB at a time, encrypts that chunk, sends it, and waits for an acknowledgement.

V. The Roadmap (Execution Plan)

This is a 4-week sprint.

Week 1: The Foundation (Console Only)

- **Goal:** No GUI. Just Python scripts communicating in the terminal.
- **Tasks:**
    - Member 2 sets up the TCP connection (Server/Client).
    - Member 3 implements the "Packetizer" (sending `MSG|Hello` and parsing it).
    - Member 1 sketches the UI wireframes on paper/Figma.

Week 2: The Interface & Discovery

- **Goal:** A working GUI that finds other computers.
- **Tasks:**
    - Member 1 builds the `CustomTkinter` layout.
    - Member 2 builds the UDP Broadcast (Auto-discovery).
    - **Integration:** Connect the UDP listener to the UI Sidebar. Now, when you open the app, it should pop up names of other teammates automatically.

Week 3: The Heavy Lifting (Files & Encryption)

- **Goal:** Secure data transfer.
- **Tasks:**
    - Member 3 adds the Encryption layer to the Network send/receive functions.
    - Member 3 writes the "File Send" logic (breaking files into chunks).
    - Member 1 adds the "Select File" button and Progress Bar to the UI.

Week 4: Polish & "The Demo"

- **Goal:** Creating the Product.
- **Tasks:**
    - **Bug Fixing:** What happens if I disconnect suddenly? Handle the crash.
    - **The Secret Mode:** Connect the "Self-Destruct" logic.
    - **PyInstaller:** Compile to `.exe`.
    - **Logo/Icon:** Add a cool custom icon so it doesn't look like a script.

VI. "How to Do It" (The Secret Sauce)

To ensure you succeed, follow these rules:

1. The "Keep-Alive" Rule The hardest part of this project is the GUI "freezing."

- *Rule:* **NEVER** run `socket.recv()` or `time.sleep()` on the main thread. Always wrap them in `threading.Thread(target=my_function, daemon=True).start()`.

2. The "Buffer" Rule When sending files, network packets can get stuck together (TCP Coalescing).

- *Rule:* Member 3 must ensure that the Receiver reads *exactly* the number of bytes specified in the Header size, no more, no less.

3. The Demo Environment For the final presentation:

- Bring a **Wi-Fi Router** (even an old one). Do not rely on College Wi-Fi (it often blocks Peer-to-Peer communication).
- Connect all 3 laptops to *your* router.
- Start the app.
- **Show, don't tell.** Click "Secret Mode," send a message, and count down "3... 2... 1..." as it disappears from the screen.

Ashish — Architect (UI & Integration)

Short: Build the CustomTkinter interface, show active peers, render chat bubbles, file picker & progress, self-destruct UI, and load/save chat history via the Logic Layer.

Week-by-week breakdown

Week 1 (Foundation)

- Day 1–2: Create project skeleton `src/ui.py`, install `customtkinter`. Build a static mock layout (sidebar, main chat area, input area).
- Day 3: Implement message bubble component functions (create_bubble(sender, text, timestamp, is_mine)).
- Day 4: Create placeholder functions that push UI events into a `logic_in_queue` (e.g., `send_text_event(payload)`).
- Day 5: Wire a `root.after(100, poll_recv_queue)` to poll incoming message queue and render messages.

Week 2 (Interface & Discovery)

- Day 1–2: Implement Active Peers sidebar: a listbox or frame of buttons. Add callback on click to open chat with selected peer.
- Day 3: Connect to discovery updates: implement `update_peers_list(peer_info)` that the Logic Layer will call (or push to `ui_queue`).
- Day 4–5: Add encryption toggle UI, display encryption status in status bar.

Week 3 (Files & Progress)

- Day 1: Add file select button (use `tk.filedialog.askopenfilename()`), emit a `send_file` event to Logic layer with file path.
- Day 2: Add transfer progress bar component and a small cancel button.

- Day 3–5: Display file progress updates by polling a `file_progress_queue` provided by Logic.

Week 4 (Polish & Demo)

- Day 1–2: Implement Self-Destruct UI: when message metadata includes self_destruct, display a countdown label attached to the bubble that updates every second with `after()`.
- Day 3: On countdown finish call `logic_in_queue.put({'cmd':'delete_msg', 'id':msg_id})` and `bubble.destroy()`.
- Day 4: Create final user-facing status bar: `Connected to X | Encryption: ON | Port: 5000`.
- Day 5: Final polish: icon, fonts, color scheme, test on target screen resolution.

Daily micro-tasks (example a single day)

- 09:00–10:30: Implement message bubble creation and basic styles.
- 10:30–11:00: Integrate `recv_queue` polling and print incoming messages.
- 11:00–12:30: Add input box and send button that places message into `send_queue`.
- 13:30–15:00: Test sending local messages (simulate with a test function).
- 15:00–17:00: Fix layout issues and prepare demo screenshots.

Code & integration tips

- Don't run network code on the main thread. UI only sends/receives via `queue.Queue`.
- `poll_recv_queue` should call `root.after(100, poll_recv_queue)` and only process a few items per call.
- For chat bubbles use `CTkFrame` + `CTkLabel` and `pack(anchor='e')` for mine, `pack(anchor='w')` for others.

- Store message metadata (message_id, timestamp, self_destruct_flag) in widget attributes (e.g., `widget.msg_id = ...`) to be able to delete later.

Testing checklist

- UI remains responsive while a 100MB file transfer runs (use dummy transfers).
- Peer list updates within ≤ 3s of discovery broadcast.
- Self-destruct countdown visually accurate and deletes UI on completion.
- Progress bar updates match real transfer bytes (within ±2%).

Pitfalls & fixes

- UI freezing during long tasks → use background threads and queues.
- Race conditions when destroying widgets → ensure all UI modifications run on main thread via `root.after`.
- Message duplication when reloading history → ensure DB load deduplicates or UI clears before fill.

Acceptance criteria

- Can send/receive messages via Logic mock (queues) without network.
- Active peers list populates from discovery events.
- File selection and progress UI functions and cancel works.
- Self-destruct removes message visually and emits delete command.

Member B — Network Engineer (Transport)

Short: Build the TCP server/client, the UDP discovery beacon & listener, and connection lifecycle (heartbeats, reconnects). Provide clean socket APIs to the Logic Layer.

Week-by-week breakdown

Week 1 (Foundation)

- Day 1–2: Implement TCP server skeleton: accept connections and spawn `handle_client(sock, addr)` in a thread.
- Day 3: Implement TCP client function `connect_to(ip, port)` returning a connected socket or raising error.
- Day 4: Create a simple console echo test between server and client on localhost using the packetizer (Member C will provide packetizer APIs).
- Day 5: Add logging and basic error handling.

Week 2 (Discovery & Integration)

- Day 1–2: Implement UDP broadcaster thread:
    - Every 2s send `DSC` packet to broadcast address on port 5556 containing `{'user':name,'port':tcp_port}`.
- Day 3–4: Implement UDP listener thread:
    - Bind UDP socket to `('' , 5556)` and call `logic_in_queue.put({'cmd':'peer_seen','ip':ip,'data':payload})` on announcements.
- Day 5: Integrate with UI through Logic Layer: ensure UI updates when new peer seen.

Week 3 (Robustness & File transfer hooks)

- Day 1: Implement heartbeat thread to send `META|0|PING` every 10s to active TCP peers; expect `PONG`.
- Day 2–4: Implement server-side per-connection receiver loop using `recv_exact`; on header read, pass parsed (type,payload) tuples to the Logic Layer via `recv_queue`.
- Day 5: Add per-connection send queue managed by a sender thread (so Logic Layer just enqueues bytes).

Week 4 (Polish & Firewall)

- Day 1–2: Implement graceful shutdown and reconnect backoff logic.
- Day 3: Prepare clear instructions for opening firewall/ports and optionally provide helper scripts (do NOT auto-run).
- Day 4–5: Stress test with multiple simultaneous connections and large file sends.

Daily micro-tasks

- 09:00 Implement `tcp_server.start(listen_port)`.
- 11:00 Implement threading model: one thread per client for recv, one thread per client for send.
- 14:00 Implement UDP broadcast content encoding and error handling.
- 16:00 Run local integration test: discovery triggers and server accept.

Code & implementation tips

- Use non-blocking design: sender thread reads from `send_queue` and calls `sock.sendall(pkt)`.
- Implement `recv_exact(sock, n)` in `protocol.py` and import it.
- For UDP broadcasting use `s.setsockopt(SOL_SOCKET, SO_BROADCAST, 1)`.

- Maintain a `peers` dict keyed by (`ip`, `tcp_port`) with last_seen timestamp and status.

Testing checklist

- Discovery messages are visible to all peers in same LAN within ≤2s.
- `recv_exact` returns exact sizes for many sizes (1B, 1KB, 1MB).
- Heartbeats detect dead peers in ≈30s and remove them.
- Server tolerates multiple simultaneous clients (>4) without crashing.

Pitfalls & fixes

- NAT/Router firewall blocks broadcast → instruct demo to use a dedicated router or enable AP isolation off.
- Port already in use → add config to choose dynamic TCP port and expose it in discovery payload.
- Blocking in `recv` → always use `recv_exact` in background thread.

Acceptance criteria

- Works end-to-end locally: discovery → connect → send TXT packet → receive packet parsed.
- Sender/Receiver threads use queues; API for Logic Layer is `send_bytes(peer, bytes)` and `recv_queue` gets `(peer, type, payload)`.

Member C — Data Engineer (Security & Protocol)

Short: Implement packet format, encryption vault (Fernet or AESGCM), recv_exact, file chunking with ACKs, and DB integration helpers for storing messages.

Week-by-week breakdown

Week 1 (Packetizer foundation)

- Day 1: Implement `make_header`, `parse_header` and `recv_exact`.
- Day 2: Write unit tests for header sizes, header parsing edge cases (malformed header, wrong length).
- Day 3–4: Implement `build_packet(type, payload_bytes, encrypted_flag)` (returns header+payload).
- Day 5: Provide example of building TXT packet and parsing it in test harness.

Week 2 (Integration with Net & UI)

- Day 1–2: Provide small API docs for Members A & B describing `send_queue` and `recv_queue` message formats and how to call `build_packet`.
- Day 3–5: Implement de/serialization helpers for message metadata (JSON wrappers for message metadata like sender, timestamp, id).

Week 3 (Encryption & Files)

- Day 1–2: Implement encryption vault:
    - `generate_key()`, `load_key(path)`, `encrypt(data, key)`, `decrypt(data, key)`.
    - Default: use Fernet for simplicity. Document AES-GCM alternative if strict AES-256 required.
- Day 3–5: Implement file chunking with per-chunk ACK:
    - Chunk format: payload = `SEQ|{seq}|NAME|{filename}\n` + chunk_bytes.
    - On receiver, reassemble by seq into temp file, send `ACK|{seq}` packet back.

    - After final EOF packet, rename temp file to final name and emit event to Logic/UI.

Week 4 (Polish & DB)

- Day 1–2: Implement saving messages to sqlite via `db.save_message(...)` (Member A will call this from Logic on send/recv).
- Day 3: Implement self-destruct flag handling: when packet contains `SELFDESTRUCT|10` include ID in DB, and delete after countdown if UI requests it.
- Day 4–5: Add thorough unit & integration tests for encryption roundtrip, file reassembly, ACK reliability.

Daily micro-tasks

- 09:00 Implement `recv_exact`.
- 11:00 Unit tests for header parse edge-cases.
- 14:00 Implement `encrypt_if_needed` and `decrypt_if_needed`.
- 16:00 Run sample encrypted TXT send/recv between two local sockets.

Code & secure tips

- Use `cryptography.Fernet` default for authenticated symmetric encryption — it's safe and reduces mistakes.
- Encrypt only the payload; header stays plain so receiver can know `enc` flag. (Header is small and unauthenticated; payload authentication is provided by Fernet.)
- For file chunk ACKs: implement a small `ACK` message type: header `ACK|0|0000000007|` + payload `SEQ:0000001`. This keeps ACK handling simple.
- Use atomic file writes: write to `filename.tmp` and rename after completion to avoid partial file exposure.

Testing checklist

- Encryption roundtrip: `encrypt(decrypt(data)) == data` for various sizes.
- Corrupted payload triggers Fernet exceptions — handle and send `ERR` packet to peer.
- File transfer reassembles perfectly for binary images and large files (test 10MB, 100MB).
- Simulate packet loss by killing client mid-transfer and ensure partial file cleanup.

Pitfalls & fixes

- Mistaking header size: always use constant `HEADER_LEN` (=17) across modules.
- Forgetting to unpad or incorrectly parse size → use zero-padded fixed width.
- Fernet size increase: encrypted payload size > original — ensure header size reflects encrypted payload length before sending.

Acceptance criteria

- `build_packet` + `parse_header` work and measured by unit tests.
- Encrypted messages decrypt correctly, and corruption is detected.
- File chunking with ACKs completes and results in perfect reassemblies.

Member D — QA / DevOps / Build & Demo

Short: Build system, packaging, firewall guidance, CI, tests, and demo readiness. Be the QA gatekeeper.

Week-by-week breakdown

Week 1

- Day 1–2: Create repo structure, `requirements.txt`, and base `README.md`.
- Day 3–4: Configure unit test runner (pytest) and basic CI (GitHub Actions stub).
- Day 5: Document dev environment setup steps.

Week 2

- Day 1–2: Create integration test harness that runs server, a client, and a mock UI to simulate flows.
- Day 3–5: Start collecting assets and icons, pick app name/logo.

Week 3

- Day 1–2: Create PyInstaller spec and initial build scripts for Windows and Linux.
- Day 3–5: Prepare packaging README and automated build script (shell/batch to run PyInstaller).

Week 4

- Day 1–2: Final build for demo machines; test the exe on each demo laptop.
- Day 3: Prepare demo checklist and fallback scripts (e.g., run console server & client).
- Day 4–5: Run full dress rehearsal on isolated router; capture issues and produce a post-mortem fix list.

Daily micro-tasks

- 09:00 Write CI test that runs a subset of unit tests.
- 11:00 Build one Windows exe with PyInstaller; record warnings.
- 14:00 Run the exe on a clean Windows VM and verify discovery works.

Packaging & firewall tips

- Do not modify Windows firewall silently. Provide clear instructions and one-click copyable commands shown in UI for admins.
- For PyInstaller include files via `--add-data` for icons and DB templates.
- Test both one-file and one-dir builds (one-file often has startup overhead).

QA testcases to automate

- Unit tests for protocol functions.
- Integration test: server + two clients sending TXT & small file.
- Stress test: 10 concurrent small transfers.
- GUI smoke test (manual): discovery, send, file, self-destruct.

Pitfalls & fixes

- Missing pack-in dependencies: include `cryptography` wheels for Windows build.
- UAC/Admin prompts: document clearly and provide commands to run as admin for firewall rules.

Acceptance criteria

- Build reproducible .exe that runs on demo laptops.
- Demo checklist passes on rehearsal with no blocking issues.

Cross-member integration: how to coordinate (must follow)

1. **Queues & APIs** — agree early on:
    - `logic_in_queue`: UI → Logic.
    - `send_queue` per peer: Logic → Network sender thread.
    - `recv_queue`: Network → Logic → UI.
    - `file_progress_queue`: Network/File → Logic → UI.
2. **Message tuple conventions**:
    - For recv_queue: `(peer_id, pkt_type, payload_bytes, extra_meta_dict)`
    - For send_queue: `{'peer': peer_id, 'bytes': b'...'}` or direct `sock` depending on impl.
3. **Header constant**: put `HEADER_LEN = 17` and `CHUNK_SIZE = 4096` in a shared `protocol.py` and import everywhere.
4. **Daily standups**: 15 minutes each day to surface integration problems early.

🧱 SecureLink Packet Structure

```
| HEADER (32 bytes) | PAYLOAD (variable) |
```

🔐 HEADER FORMAT (32 BYTES – FIXED)

Offset	Size	Field Name	Description
0	4 bytes	MAGIC	Protocol signature (`SLNK`)

4	1 byte	VERSION	Protocol version (e.g., 0x01)
5	1 byte	TYPE	Message type
6	1 byte	FLAGS	Encryption, Self-destruct, ACK
7	1 byte	RESERVED	For future use
8	8 bytes	MESSAGE_ID	Unique message identifier
16	8 bytes	TIMESTAMP	Unix timestamp
24	4 bytes	PAYLOAD_SIZE	Size of payload in bytes
28	4 bytes	CHECKSUM	CRC32 / integrity check

Total: 32 bytes (Fixed)

🧠 FIELD DEFINITIONS (VERY IMPORTANT)

1️⃣ MAGIC (4 bytes)

```
b'SLNK'
```

✔ Verifies packet belongs to SecureLink
✔ Rejects garbage or malicious packets
✔ Very common in real protocols (PNG, ELF, TCP)

2️⃣ VERSION (1 byte)

```
0x01
```

✔ Allows future upgrades
✔ Backward compatibility
✔ Professors LOVE this

3️⃣ TYPE (1 byte)

Value	Meaning
0x01	TXT (Text message)
0x02	FIL (File chunk)
0x03	DSC (Discovery)
0x04	ACK (Acknowledgment)
0x05	META (Typing, Read receipts)
0x06	ERR (Error)

✔ Faster than strings
✔ Cleaner parsing
✔ Real protocol style

4️⃣ FLAGS (1 byte – Bitmask)

Each bit has meaning:

Bit	Meaning
0	Encrypted
1	Self-destruct
2	Is ACK
3	Is Last File Chunk
4–7	Reserved

Example:

```
FLAGS = 0b00001001
```

→ Encrypted + Last Chunk

✔ EXTREMELY powerful
✔ One byte = many features
✔ Very impressive in viva

5️⃣ MESSAGE_ID (8 bytes)

`uint64`

✔ Tracks message uniqueness
✔ Required for ACKs, retries, self-destruct
✔ Used in WhatsApp/Signal-like systems

6️⃣ TIMESTAMP (8 bytes)

`Unix timestamp (milliseconds)`

✔ Ordering messages
✔ Replay attack protection
✔ DB synchronization

7️⃣ PAYLOAD_SIZE (4 bytes)

`uint32`

✔ Receiver knows exactly how many bytes to read
✔ Solves TCP packet coalescing issue

8️⃣ CHECKSUM (4 bytes)

`CRC32(payload)`

✔ Detects corruption
✔ Very strong justification in report
✔ Extra safety for file transfer

🧪 Example: Text Message Packet

```
MAGIC       : SLNK
VERSION     : 01
TYPE        : 01 (TXT)
FLAGS       : 01 (Encrypted)
MSG_ID      : 00000000000000A1
TIMESTAMP   : 0000018F3A2C8000
SIZE        : 00000012 (18 bytes)
CHECKSUM    : A3F9C210
```
